Submission — aegis-statefulset

> A reference architecture for single-master-AZ Kubernetes hosting of
> LevelDB-backed stateful applications, with cold DR via Velero,
> Strangler-Fig migration support, and observability built in.
>
> Status: proof-of-concept. Architecture is stable; implementation is skeletal
> by design — the submission covers structural depth (10 thematic ADRs (50 private predecessors) across 14
> domains) over surface coverage. Anonymised for portability.

---

1. Overview — three organising principles

Every architectural decision traces back to one of these. Every knob in
`values.yaml` is the operator's seat at one of these levers.

P1 — Data on EBS is the only irreplaceable asset; everything else is declarative

Customer state lives on encrypted EBS volumes managed via LVM. Cluster,
deployments, routing tables, container images, AZ topology — all reproducible
in 30–60 minutes from Helm + Terraform. Stateful tier gets `reclaimPolicy=Retain`,
1:1 pod-to-node, conservative backup cadence. Stateless tier gets autoscaling,
spot mix, cattle treatment.

P2 — RPO is a configurable trade-off, not a fixed floor

The challenge spec sets RPO 6h as the upper bound. The default is tighter —
5-min cadence yields typical recovery point ~30 sec — but the lever is exposed
in `values.yaml` (`backup.cadence_minutes`), and the cost-RPO curve is
documented for operator self-tune. The submission gives the curve, not a
pre-decided answer.

P3 — Conservative on recovery, aggressive on failure detection

Failover detection: 50% unhealthy / 2 min sustained — reactive, sensitive.
Recovery detection: 95% healthy / 60 min sustained — notify-only, never
auto-failback. Asymmetric thresholds prevent flap.

---

2. Hard architectural requirements (Layer 1 — platform owns)

Layer 1 decisions are non-negotiable for the customer; the platform owns them.

#	Requirement	Why
L1.1	Single master AZ for all workloads at any moment	LevelDB is single-writer; cross-AZ chatter on a stateful tier with no replication primitives is wasted latency and double-cost. (ADR-01 modified.)
L1.2	EBS encryption at rest with per-tier customer-managed KMS keys	Compliance and blast-radius isolation. (ADR-07.)
L1.3	StatefulSet with `reclaimPolicy=Retain` + claimRef pre-binding	Pod-to-PV mapping must survive cluster recreation. (ADR-02.)
L1.4	1:1 pod-to-node ratio for the stateful tier	Disk-bound workloads compete poorly for I/O; 1:1 is honest about it. (ADR-02.)
L1.5	LVM-managed EBS, online-expandable	The application's growth pattern is unpredictable; offline-resize is unacceptable downtime. (ADR-02.)
L1.6	Cross-region S3 replication of backups	Single-region durability is not a DR posture. (ADR-04.)
L1.7	Three-path DR with named RPO/RTO per path	Operators must know which path applies for which failure shape. (ADR-04.)
L1.8	Three-Layer DR — Layer 1 (Velero) cold + Layer 2 (Terraform helm_release) warm +

Layer 3 (Route 53) DNS | Each layer recovers a different failure class without contention. (ADR-04.) |
| L1.9 | SHA-pinned external image references | Supply-chain hygiene per NIST SSDF / SLSA L3. (ADR-09, ADR-09.) |
| L1.10 | OpenTelemetry-only instrumentation | Vendor neutrality at the instrumentation layer; backend is reversible. (ADR-06, ADR-06.) |

---

3. Demo-only choices (Layer 2 — platform decides for the POC)

These are honest defaults the platform picks for the POC submission. Each is re-decidable post-conversation; none is load-bearing on the architecture.

#	Choice	Reasoning
L2.1	Mock stateful app — ~30 lines Go, file-per-key under `/data`	Smallest object that exercises the operational shape (PV ownership, write-on-disk, node-loss survival). (`app/main.go`.)
L2.2	Pattern 2 LDB layout (one DB per pod) for the POC mock	Pattern 1 vs 2 is a Stage 3 question; Pattern 2 is the simpler default for the demo. (ADR-04.)
L2.3	3 NAT Gateways (one per AZ)	Cost vs blast-radius; Mittelstand-correct. Single-NAT designs are cheaper but break the AZ-isolation story we are otherwise telling.
L2.4	Cold DR via Velero + EBS snapshot	Hot-DR is structurally the wrong fit for LevelDB physics + 5-min cadence; see `docs/operations/why-cold-dr.md`. (ADR-04.)
L2.5	Master AZ rotation via `aws eks update-nodegroup-config` + Velero restore (not `terraform apply`)	Topology is constant; AZ choice is a runtime scaling decision. (ADR-01 modified, ADR-04.)
L2.6	3-tier flow: ALB → API → Envoy → StatefulSet, all in master AZ	Edge / business logic / mesh / state separation; standard senior shape.
L2.7	10 thematic ADRs (originally 50 private) — DR + LDB-layout topics consolidated into ADR-04	Submission covers structural depth, not surface coverage.

---

4. Customer configuration knobs (Layer 3 — operator decides)

The chart exposes the trade-off curve through a small, deliberate set of values. Full inventory in `helm/aegis-statefulset/values.yaml`.

Knob	Default	Range	Trade-off
`backup.cadence_minutes`	`5`	`1` – `360`	Lower = tighter RPO + higher EBS Snapshot / S3 cost; upper bound is the spec's 6 h RPO.
`ha.model`	`cold_dr`	`cold_dr`	Single mode in Wave 4. active-passive (private ADR-014) retired in favour of cold DR (see ADR-04).
`routing.backend`	`dynamodb`	`dynamodb`, `customer_supplied`	DynamoDB Global Tables is POC reference; customer can substitute any backend that satisfies the 6-property contract per ADR-03.
`stateful.cells.count`	`1`	`1` – `N`	POC default 1; production scale matches customer's existing partitioning per ADR-05 Strangler Fig migration.
`storage.ebs_size_gb`	`2048`	`512` – `16384`	Per-pod EBS size; LVM-managed and online-expandable.
`master_az`	`eu-central-1a`	any AZ in region	Master AZ for ALL workloads. Standby AZs `desiredSize=0`; rotation via `aws eks update-nodegroup-config` (ADR-01 modified, ADR-04).
`dr_region`	`eu-west-1`	any AWS region	Cross-region DR target for EBS Snapshot copy via DLM (ADR-04).
`observability.backend`	`grafana_cloud`	`grafana_cloud`, `amp_amg`	OTel-instrumented; backend reversible per ADR-06.

---

5. Architecture diagram

```mermaid
flowchart TB
    Client["Client"]

    subgraph Edge["Edge — Route 53 + ALB (regional)"]
        R53["Route 53 weighted A-record"]
        ALB["ALB (master region)"]
    end

    subgraph MasterAZ["Master AZ (eu-central-1a, default)"]
        API["API tier — stateless, autoscaled"]
        Envoy["Envoy mesh — stateless"]
        SS["StatefulSet pods — 1:1 pod-to-node"]
        EBS["EBS volumes — Retain, LVM"]
    end

    subgraph StandbyAZ["Standby AZs (b, c) — desiredSize=0"]
        StandbyNG["Node groups, scale-on-demand"]
    end

    subgraph DR["DR region (eu-west-1)"]
        Velero["Velero restore target"]
        DRALB["DR ALB (warm)"]
    end

    subgraph Backup["Backup pipeline"]
        LVM["LVM snapshot"]
        Restic["Restic"]
        S3["S3 (versioned, KMS, cross-region replicated)"]
    end

    Client --> R53 --> ALB --> API --> Envoy --> SS --> EBS
    EBS --> LVM --> Restic --> S3
    S3 -.cross-region.-> Velero
    SS -.AZ rotation.-> StandbyNG
    R53 -.region cutover.-> DRALB
```

3-tier flow detail: ALB does TLS termination + routing-by-host. API tier handles auth + business logic. Envoy provides east-west traffic shaping (retries, circuit breaks) and acts as a blue/green seam for the stateful tier. StatefulSet pods are pinned 1:1 to nodes in the master AZ.

---

6. Disaster recovery summary

Three failure shapes, three paths, each with named RPO / RTO.

Failure shape	Path	RPO	RTO	Mechanism
etcd corrupted, EBS intact	Path A	0	~30 min	EBS-tag truth → regenerate PV manifests with claimRef → Helm install at discovered count. (ADR-04, `scripts/dr/recover-cluster.sh`.)
Routing table lost, pods intact	Path B	0	~10 min	Pod inventory truth → re-derive override delta. (ADR-04, `scripts/dr/warm-routing-table.sh`.)
AZ failure (subnet, network, hardware)	AZ rotation	~5 min	~20 min	Standby AZ scale-up + Velero restore. (ADR-04, `scripts/dr/az-rotation.sh`.)
Source region lost	Path C / Region recovery	~5 min	~50 min	Cross-region cold-DR restore + Route 53 cutover. (ADR-04, `scripts/dr/region-failure-recovery.sh`.)
Backup lost too	Path C — last resort	bounded by cadence (max 6h per spec)	6–12h	Restic restore from cross-region S3. (ADR-04, `scripts/dr/restore-from-s3.sh`.)

The Three-Layer DR model (ADR-04) decouples these:
- **Layer 1 (Velero, cold)** — application namespaces.

- **Layer 2 (Terraform `helm_release`, warm)** — controllers (kube-system,
  monitoring, kyverno, ESO, ArgoCD).
- **Layer 3 (Route 53, DNS)** — traffic.

Each layer recovers a different failure class without contention.

---

7. Cost summary

Component	Monthly (default)	Notes
EKS control plane	$73	Single cluster
Stateful nodes (master AZ only)	$500	r6id.2xlarge on-demand, 1:1 pod-to-node, POC `cells.count=1`; scales linearly with cell count
Stateless nodes (api + envoy + system)	$800	Karpenter mixed spot 70% / on-demand 30%
EBS gp3 storage (master AZ)	$200	1× 2 TB primary at POC scale; warm-standby AZs charge $0 (no PVCs until rotation)
3× NAT Gateways	$99	Per-AZ blast-radius isolation; warm-standby rotation readiness (ADR-10 §6)
EBS Snapshot (5-min cadence, 30-day retention)	$300	Velero schedule + DLM lifecycle
Cross-region snapshot copy (Glacier Instant Retrieval)	$150	DLM cross-region per ADR-04
S3 (Velero metadata + cross-region replication)	$50	Backup metadata only; data lives in EBS Snapshots
Observability (Grafana Cloud Pro)	$500	OTel-instrumented; AMP/AMG alternative ~$700
Total (cold DR, 5-min cadence, POC `cells.count=1`)	**~$2,700**	Mittelstand budget; scales linearly with cell count

Cost knobs:
- Loosen cadence 5-min → 1h → save ~$200/mo, RPO worsens to ~30 min.
- Reduce to 1 NAT Gateway → save ~$66/mo, lose warm-standby rotation readiness.
- Add cells per ADR-05 migration → ~$700/mo per cell (node + EBS).

---

8. Stage 3 questions consolidated

The submission deliberately defers these to a live conversation. Each maps to a knob, an ADR caveat, or an architecture variant.

1. **Service decomposition** — actual count and identity of microservices in the legacy stack?
2. **Routing key location** — tenant identifier in URL, subdomain, or JWT?
3. **Legacy session storage** — in-memory, file, Redis, or RDBMS?
4. **Placement algorithm** — deterministic by hash, or capacity-aware with a bin-packing component?
5. **Customer SLA shape** — single RPO across all tiers, or tiered (e.g., enterprise sub-2h cross-region RTO)?
6. **Multi-pod model** — confirm per-tenant cell vs hash-sharded?
7. **Pod density distribution** — many small tenants, or enterprise-dedicated profile dominant?
8. **Failover detection threshold** — 50% unhealthy / 2 min sustained acceptable?
9. **Failback policy** — Option A (standby becomes new primary permanently) vs Option B (rebalance back in maintenance window)?
10. **Maintenance window cadence** — what frequency is acceptable for operational rebalancing?
11. **LDB layout** — Pattern 1 (one DB shared by all tenants in pod) or Pattern 2 (one DB per tenant) — drives relocation tier matrix per ADR-04.
12. **Existing tenant→cluster mapping storage** — Postgres / Aurora / internal service / Redis?
13. **App code changeability** — can we add a small telemetry metric, or must the application container stay strictly unchanged?
14. **Migration window availability** — is there a planned window for

    Strangler-Fig cutover, or is it incremental?
15. **Customer's existing GitOps + DR tooling** — ArgoCD already in use?
    Velero already in use? — drives "what we own vs what we adopt".

Each maps to a knob in `values.yaml` or a caveat in an ADR. The submission
gives the curve, not the answer.

---

9. Future work — "more time would add"

In rough priority order if the team wanted depth where the POC is shallow:

1. **WAL shipping** for sub-5-min RPO (out of POC scope; would change the
   active-passive design).
2. **Tier C relocation** — extraction code for live tenant relocation under
   Pattern 1 layout (~1 month of focused work per ADR-04).
3. **Per-tenant cost attribution dashboards** beyond the Athena queries
   in `scripts/finops/` — full Grafana panels with tenant-pivot tables.
4. **Multi-region active-active** — only if a customer SLA forces it; the
   default cold-DR + 50 min RTO is the right trade-off for Mittelstand.
5. **Service mesh upgrade** from Envoy as a sidecar to Cilium Service Mesh
   or Linkerd — east-west traffic shaping with kernel-level efficiency.
6. **Chaos automation** — schedule Phase 1 + Phase 2 drills as recurring
   GitHub Actions workflows with auto-roll-back; the scripts already exist,
   just need a scheduler.
7. **Per-tenant migration UX** — a small operator-facing CLI on top of
   `scripts/relocation/per-tenant-relocate.sh`.
8. **Backup verification automation** — periodic restore-and-checksum
   pipeline, beyond "did the snapshot land in S3".

Anything not in the POC is in this list because it is a depth choice, not a
shape choice — the architecture supports each item without re-shaping.

---

10. For reviewers — entry points

You want to read about…	Start here
The whole picture	`README.md` and this file
Why each architectural decision	`_context/adr/` (50 private predecessors of the public 10)
Why cold DR over active-passive multi-region	`docs/operations/why-cold-dr.md`
Disaster runbook	`docs/operations/region-failure-recovery.md`
Ownership boundaries	`docs/operations/scope-boundaries.md`
Per-tenant relocation tiers	`docs/operations/per-tenant-relocation.md`
Architectural assumptions	`docs/architecture-assumptions.md`
Mock app implementation	`app/main.go`
Chaos demo scripts	`scripts/chaos/`
Helm chart	`helm/aegis-statefulset/`
Terraform	`infrastructure/terraform/`

Submission status: anonymised, public, intentionally portable. The
architecture is reusable as a reference for stateful K8s workloads at
Mittelstand-scale B2B SaaS, regardless of whether this particular
conversation converts.